

3.50 mselstr 文字列による行選択

f=で指定した項目の値が、v=で指定した文字列に一致すれば、その行を選択する。

典型例を表 3.19～3.21 に示す。表 3.20 では key に関係なく val が"y"である行を選択する。表 3.21 では、val が"x"の行を含んでいる同一キーの行全て、すなわち key 項目が"a"である行全てを選択する。すなわち key 項目が"b"の行はいずれも"x"を含んでいないので選択されない。

表 3.19 入力データ

key	val
a	x
a	y
b	y
b	z

表 3.20 f=val v=y

key	val
a	y
b	y

表 3.21 k=key f=val v=x

key	val
a	x
a	y

また、以下に示すように多様な選択条件を指定することも可能である。このコマンドで指定できない複雑な条件 (例えば正規表現など) を設定するのであれば **msel** コマンドを利用すればよい。

- v=に複数の文字列を指定すれば、いずれかの文字列にマッチすれば選択される。
- f=に複数項目を指定すれば、いずれかの項目の値がマッチすれば選択される。
- 複数項目のマッチ条件を AND 条件とすることも可能 (-F オプション)。
- 完全一致だけでなく、先頭一致、末尾一致、部分一致の指定も可能 (-head, -tail, -sub オプション)。
- k=を指定することでキー単位で選択することが可能。
- キー単位選択の場合、複数レコードの AND 条件を指定可能 (-R オプション)。

いま同じキーのデータとして2項目2行からなるデータ (表 3.22) に対して、

```
mselstr k=key f=fld1,fld2 v=s1,s2
```

を実行した場合、オプション-R, -F の指定の有無によるマッチ条件を表 3.23 に示す。

表 3.22 入力データ

key	fld1	fld2
k	v _{a1}	v _{a2}
k	v _{b1}	v _{b2}

表 3.23 表 3.22 で示されるデータに、mselstr k=key f=fld1,fld2 v=v1,v2 を実行した時の、-R,-F オプションの指定の有無によるマッチ条件の違い。条件にマッチすれば全行 (2 行) 出力され、アンマッチなら 1 行も出力されない。

-F	-R	マッチ条件
		$((v_{a1} == s1 \text{ or } v_{a1} == s2) \text{ or } (v_{a2} == s1 \text{ or } v_{a2} == s2)) \text{ or } ((v_{b1} == s1 \text{ or } v_{b1} == s2) \text{ or } (v_{b2} == s1 \text{ or } v_{b2} == s2))$
-F		$((v_{a1} == s1 \text{ or } v_{a1} == s2) \text{ and } (v_{a2} == s1 \text{ or } v_{a2} == s2)) \text{ or } ((v_{b1} == s1 \text{ or } v_{b1} == s2) \text{ and } (v_{b2} == s1 \text{ or } v_{b2} == s2))$
	-R	$((v_{a1} == s1 \text{ or } v_{a1} == s2) \text{ or } (v_{a2} == s1 \text{ or } v_{a2} == s2)) \text{ and } ((v_{b1} == s1 \text{ or } v_{b1} == s2) \text{ or } (v_{b2} == s1 \text{ or } v_{b2} == s2))$
-F	-R	$((v_{a1} == s1 \text{ or } v_{a1} == s2) \text{ and } (v_{a2} == s1 \text{ or } v_{a2} == s2)) \text{ and } ((v_{b1} == s1 \text{ or } v_{b1} == s2) \text{ and } (v_{b2} == s1 \text{ or } v_{b2} == s2))$

書式

```
mselstr f= v= [k=] [u=] [-F] [-r] [-R] [-sub] [-head] [-tail] [-W] [i=] [o=] [bufcount=]
[-assert_diffSize] [-assert_nullkey] [-assert_nnullin] [-nfn] [-nfno] [-x] [-q] [tmpPath=] [--help] [--help1]
```

[--version]

パラメータ

f= 検索対象となる項目名リスト (複数項目指定可) を指定する。
v= **f=**パラメータで指定した項目の値が、ここで指定した文字列リスト (複数項目指定可) の1つにマッチすれば選択される。
k= 選択する単位となるキー項目 (複数項目指定可) を指定する。
o= 指定の条件に一致する行を出力するファイル名を指定する。
u= 指定の条件に一致しない行を出力するファイル名を指定する。
-F **f=** パラメータで複数項目を指定した場合、その全ての値がマッチする行を撰択する。
-r 条件反転
 選択ではなく削除する。
-R **k=** パラメータを指定した場合、その全ての行がマッチすれば行を撰択する。
-sub 検索を完全一致ではなく部分文字列マッチで比較する。
 すなわち、**f=**パラメータで指定した項目の値に、
v=パラメータで指定の文字列が部分文字列として含まれていればその行を撰択する。
-head 先頭文字列マッチオプション
-tail 末尾文字列マッチオプション
-W **-sub,-head,-tail** オプションが指定されているときにワイド文字として部分文字列マッチをおこなう。

利用例

例 1: 基本例

「商品」項目の値が **apple**、**orange** に完全一致する行を選択し、**rs11.csv** に出力する。**u=oth1.csv** を指定すれば、それ以外の行は **oth1.csv** に出力する。**pineapplejuice** は完全一致ではないので、**oth1.csv** に出力される。

```
$ more dat1.csv
商品,金額
apple,100
milk,350
orange,100
pineapplejuice,500
wine,1000
$ mselstr f=商品 v=apple,orange u=oth1.csv i=dat1.csv o=rs11.csv
#END# kgselstr f=商品 i=dat1.csv o=rs11.csv u=oth1.csv v=apple,orange
$ more rs11.csv
商品,金額
apple,100
orange,100
$ more oth1.csv
商品,金額
milk,350
pineapplejuice,500
wine,1000
```

例 2: 行の削除

-r オプションを指定することで、例 1 とは逆に、商品項目の値が **apple**、**orange** に完全一致する行を削除し、**rs12.csv** に出力する。

```
$ mselstr f=商品 v=apple,orange -r i=dat1.csv o=rs12.csv
#END# kgselstr -r f=商品 i=dat1.csv o=rs12.csv v=apple,orange
$ more rs12.csv
商品,金額
```

```

milk,350
pineapplejuice,500
wine,1000

```

例 3: キー単位での選択

orange を購入したことのある顧客を選択する k=顧客を指定することで、orange を購入したことのある顧客の他に購入した商品の行を含めて選択する。それ以外の行は oth2.csv に出力する。

```

$ more dat2.csv
顧客, 商品, 金額
A,apple,100
A,milk,350
B,orange,100
B,orange,100
B,pineapple,500
B,wine,1000
C,apple,100
C,orange,100
$ mselstr k=顧客 f=商品 v=orange u=oth2.csv i=dat2.csv o=rsl3.csv
#END# kgselstr f=商品 i=dat2.csv k=顧客 o=rsl3.csv u=oth2.csv v=orange
$ more rsl3.csv
顧客 %, 商品, 金額
B,orange,100
B,orange,100
B,pineapple,500
B,wine,1000
C,apple,100
C,orange,100
$ more oth2.csv
顧客 %, 商品, 金額
A,apple,100
A,milk,350

```

例 4: 部分一致

「商品」項目の値が apple に部分一致するの行を選択し、rsl4.csv に出力する。部分一致であるため pine(apple)juice も rsl4.csv に出力される。

```

$ mselstr f=商品 v=apple -sub i=dat1.csv o=rsl4.csv
#END# kgselstr -sub f=商品 i=dat1.csv o=rsl4.csv v=apple
$ more rsl4.csv
商品, 金額
apple,100
pineapplejuice,500

```

例 5: ワイド文字の部分一致

「商品」項目の値がワイド文字の「柿」、「桃」、「葡萄」の行を選択 (部分一致) 選択項目にワイド文字が使用されている場合にバイト単位のマッチングを使用すると、マルチバイト文字をまたいだ文字列にマッチングする可能性がある。その為、ワイド文字が選択項目に含まれる場合は -W オプションを使用して、ワイド文字を使用していることを意図的に示す必要がある。

```

$ more dat3.csv
商品, 金額
果物: 柿, 100
果物: 桃, 250
果物: 葡萄, 300

```

```

果物:梨,450
果物:苺,500
$ mselstr f=商品 v=柿,桃,葡萄 -sub -W i=dat3.csv o=rsl5.csv
#END# kgselstr -W -sub f=商品 i=dat3.csv o=rsl5.csv v=柿,桃,葡萄
$ more rsl5.csv
商品,金額
果物:柿,100
果物:桃,250
果物:葡萄,300

```

例 6: 商品の購入日と前回の購入日が 2013 年の商品データを選択

-F オプションを指定することで、同じ商品を 2013 年以内に購入したことのある (購入日と前回購入日両方が 2013 年) 商品行を選択し、rsl6.csv に出力する。それ以外の行は oth3.csv に出力する。

```

$ more dat4.csv
顧客,商品,金額,性別,購入日,前回購入日
A,apple,100,1,2013/01/04,2013/01/01
A,milk,350,1,2013/04/04,2011/05/06
B,orange,100,2,2012/11/11,2011/12/12
B,orange,100,2,2013/05/30,2012/11/11
B,pineapple,500,2,2013/04/15,2013/04/01
B,wine,1000,2,2012/12/24,2011/12/24
C,apple,100,2,2013/02/14,NULL
C,orange,100,2,2013/02/14,2013/01/31
D,orange,100,2,2011/10/28,NULL
$ mselstr f=購入日,前回購入日 -F -sub v=2013 u=oth3.csv i=dat4.csv o=rsl6.csv
#END# kgselstr -F -sub f=購入日,前回購入日 i=dat4.csv o=rsl6.csv u=oth3.csv v=2013
$ more rsl6.csv
顧客,商品,金額,性別,購入日,前回購入日
A,apple,100,1,2013/01/04,2013/01/01
B,pineapple,500,2,2013/04/15,2013/04/01
C,orange,100,2,2013/02/14,2013/01/31
$ more oth3.csv
顧客,商品,金額,性別,購入日,前回購入日
A,milk,350,1,2013/04/04,2011/05/06
B,orange,100,2,2012/11/11,2011/12/12
B,orange,100,2,2013/05/30,2012/11/11
B,wine,1000,2,2012/12/24,2011/12/24
C,apple,100,2,2013/02/14,NULL
D,orange,100,2,2011/10/28,NULL

```

例 7: 商品の購入日と前回の購入日が 2013 年の顧客データの抽出

k=顧客を指定することで、同じ商品を 2013 年以内に購入したことのある顧客の他に購入した商品の行を含めて選択する。それ以外の行は oth4.csv に出力する。

```

$ mselstr k=顧客 f=購入日,前回購入日 -F -sub v=2013 u=oth4.csv i=dat4.csv o=rsl7.csv
#END# kgselstr -F -sub f=購入日,前回購入日 i=dat4.csv k=顧客 o=rsl7.csv u=oth4.csv v=2013
$ more rsl7.csv
顧客 %,商品,金額,性別,購入日,前回購入日
A,apple,100,1,2013/01/04,2013/01/01
A,milk,350,1,2013/04/04,2011/05/06
B,orange,100,2,2012/11/11,2011/12/12
B,orange,100,2,2013/05/30,2012/11/11
B,pineapple,500,2,2013/04/15,2013/04/01
B,wine,1000,2,2012/12/24,2011/12/24
C,apple,100,2,2013/02/14,NULL
C,orange,100,2,2013/02/14,2013/01/31
$ more oth4.csv
顧客 %,商品,金額,性別,購入日,前回購入日

```

```
D,orange,100,2,2011/10/28,NULL
```

例 8: 2013 年度の新規顧客情報の抽出

-R オプションを指定することで、購入日、前回購入日両方が 2013 年,NULL(前回購入なし) の顧客情報を抽出する。
つまり 2013 年の新規顧客データを選択し、rs18.csv に出力する。それ以外の行は oth5.csv に出力する。

```
$ mselstr k=顧客 f=購入日, 前回購入日 -F -R -sub v=2013,NULL u=oth5.csv i=dat4.csv o=rs18.csv
#END# kgselstr -F -R -sub f=購入日, 前回購入日 i=dat4.csv k=顧客 o=rs18.csv u=oth5.csv v=2013,NULL
$ more rs18.csv
顧客 %0, 商品, 金額, 性別, 購入日, 前回購入日
C,apple,100,2,2013/02/14,NULL
C,orange,100,2,2013/02/14,2013/01/31
$ more oth5.csv
顧客 %0, 商品, 金額, 性別, 購入日, 前回購入日
A,apple,100,1,2013/01/04,2013/01/01
A,milk,350,1,2013/04/04,2011/05/06
B,orange,100,2,2012/11/11,2011/12/12
B,orange,100,2,2013/05/30,2012/11/11
B,pineapple,500,2,2013/04/15,2013/04/01
B,wine,1000,2,2012/12/24,2011/12/24
D,orange,100,2,2011/10/28,NULL
```

関連コマンド

msel : より複雑な条件で行選択を行う。

mcommon : 選択対象となる文字列の数が多いときは、参照ファイルを用意することで mcommon コマンドが使える。